WORKTRACE

The Competency Layer for the AI-Native Workforce

Software Requirements & Workflow Specification

Prepared for: OpenAI Build Week Challenge — Education Track

Prepared by: Yusra Tariq

Built with: Codex (GPT-5.6)

WORKTRACE is a simulated workplace environment in which learners collaborate with an AI teammate to complete a real engineering task. Instead of grading only the final output, WORKTRACE instruments and evaluates the learner's process — what they asked AI, which suggestions they accepted or rejected, whether they verified claims, and whether they can independently justify the final decision. The output is a Competency Receipt: portable, evidence-backed proof of how a person actually works with AI.

1. Problem Statement

AI can now generate polished code, essays, and analysis on demand. This has broken the traditional signal universities, bootcamps, and employers rely on: a finished artifact used to imply the person behind it understood and could reproduce the reasoning. That assumption no longer holds.

Evaluators currently have no reliable way to answer:

- Did this person understand the work, or did the AI do the thinking?
- Can they detect when the AI is confidently wrong?
- Do they verify claims, or accept AI output uncritically?
- Can they independently explain and defend the final decision?

This gap affects universities issuing grades, bootcamps issuing certificates, and employers evaluating candidates and interns — all currently guessing at a problem that did not exist three years ago.

2. Solution Overview

WORKTRACE places a learner inside a simulated company with one realistic mission (e.g. "Checkout conversion has dropped 12% — investigate and propose a fix"). The learner has access to a real codebase, project data, an AI teammate, and simulated stakeholders (a Manager and a skeptical Reviewer). Codex plays three roles simultaneously:

- **Codebase Generator** — produces the mission's starting project and realistic data.
- **AI Teammate** — assists the learner, and occasionally offers a plausible but incorrect suggestion the learner must catch.
- **Evaluator** — observes every interaction and scores the learner's judgment, not just their code.

Every AI interaction is logged as a decision event: suggestion offered, accepted, rejected, or verified. At the end of the mission, WORKTRACE generates a Competency Receipt — a scored, evidence-backed summary of how the learner worked with AI.

3. Target Users

- **Primary:** CS students and bootcamp graduates who need to prove AI-era competency beyond a finished project or a GPA.
- **Secondary:** Bootcamps and universities that need a credible way to certify AI-native skill, not just course completion.
- **Tertiary:** Employers and recruiters screening candidates who want evidence of judgment, not just AI-polished portfolios.

4. Functional Requirements

Each requirement is tagged with an ID, priority (Must / Should / Could — MoSCoW), and the Build Week scope it falls in.

ID	Requirement	Description	Priority
FR-1	Mission Generation	System generates one fixed mission (checkout conversion drop) with a starter codebase, seed data, and a written problem brief via Codex at session start.	Must
FR-2	Workspace Environment	Learner is given a code view (file tree + editor/viewer), a chat panel to talk to the AI teammate, and an inbox for Manager/Reviewer messages.	Must
FR-3	AI Teammate Chat	Learner can ask the AI teammate questions about the codebase, data, or approach; Codex responds contextually using the mission state.	Must
FR-4	Seeded Incorrect Suggestion	At least once per session, the AI teammate offers a plausible but flawed suggestion (e.g. an inefficient fix or a wrong data read) for the learner to evaluate.	Must
FR-5	Decision Logging	Every AI interaction is captured as an event: suggestion shown, accepted, rejected, or modified, with a timestamp and short rationale field.	Must
FR-6	Verification Prompt	System periodically asks the learner to confirm or challenge an AI claim (e.g. "the AI says X caused the drop — do you agree?") and logs the response.	Must
FR-7	Manager / Reviewer Messages	Simulated Manager and Reviewer personas send Codex-generated messages that react to learner actions (e.g. a deadline nudge, a skeptical follow-up question).	Should
FR-8	Solution Submission	Learner submits their final fix/proposal along with a short written justification of their decision.	Must
FR-9	Independent Explanation Check	System asks the learner one follow-up question about their submitted solution and evaluates whether the explanation is coherent and independent of AI phrasing.	Should
FR-10	Competency Scoring	Codex scores the session across five dimensions (Technical Execution, Problem Framing, AI Verification, Independent Judgment, Communication) based on logged events.	Must
FR-11	Competency Receipt Generation	System renders a final results screen: scores, evidence counts (decisions made, suggestions rejected, verifications performed), and a short narrative summary.	Must
FR-12	Receipt Export	Learner can download or share the Competency Receipt as a shareable artifact (image or PDF).	Should
FR-13	Session Replay	Learner can scroll back through their own decision log after the mission ends to review what happened and when.	Could

ID	Requirement	Description	Priority
FR-14	Adaptive Difficulty	Future: mission difficulty (codebase size, ambiguity, number of flawed suggestions) adjusts to learner's demonstrated skill level.	Could — post-hackathon

5. Non-Functional Requirements

Category	Requirement
Performance	AI teammate responses return within 5–8 seconds under normal load; mission generation completes within 20 seconds at session start.
Usability	A first-time user must be able to understand the mission and take a first action within 2 minutes, without a tutorial.
Reliability	Decision logging must not lose events; if the AI teammate call fails, the session should degrade gracefully with a retry rather than losing session state.
Consistency	The AI teammate and Manager/Reviewer personas must stay contextually consistent within a single session (no contradicting earlier codebase or mission facts).
Security & Privacy	No real user data is used; mission data is synthetic. API keys and credentials are server-side only, never exposed to the client.
Scalability	Architecture supports one concurrent mission per learner for the hackathon build; designed so additional mission types can be added without core rework.
Portability	Competency Receipt is exportable as a static artifact (image/PDF) so it can be shared outside the platform (e.g. LinkedIn, resume, recruiter link).
Auditability	Every score on the Competency Receipt must be traceable back to a specific logged decision event — no opaque or unexplained scoring.
Maintainability	Mission content (brief, codebase, personas) is defined in a structured format, not hardcoded, so new missions can be authored without touching core logic.
Demo Reliability	Core flow (mission → AI interaction → flawed suggestion → rejection → submission → receipt) must work end-to-end offline of network flakiness for the live demo.

6. System Architecture

WORKTRACE is composed of four cooperating layers. Codex is invoked at three distinct points in the flow, each doing structurally different work — not one generic prompt-response loop.

Layer	Responsibility	Codex Role
Mission Layer	Defines the fixed scenario: company, role, starter codebase, seed data, problem brief.	Generates codebase, data, and brief once at session start.
Interaction Layer	Chat interface between learner and AI teammate; inbox for Manager/Reviewer messages.	Acts as AI teammate; generates one seeded flawed suggestion; personas react to learner actions.
Instrumentation Layer	Captures every AI interaction as a structured decision event (suggestion, accept/reject/modify, verification response).	Not Codex — deterministic logging layer that feeds the evaluator.
Evaluation Layer	Scores the full decision log across five competency dimensions; generates the Competency Receipt.	Evaluates session transcript + decision log; produces scores and narrative summary.

High-Level Data Flow

Learner joins mission → Codex generates codebase + brief → Learner investigates and chats with AI teammate → AI teammate offers help (including one seeded flawed suggestion) → Instrumentation layer logs every accept / reject / verify event → Learner submits solution + justification → Codex evaluates the full log and transcript → Competency Receipt is rendered and made exportable.

7. End-to-End Workflow

The workflow below is scoped to the single hackathon demo mission: "Checkout conversion has dropped 12% at NovaCommerce."

Step 1 — Onboarding

Learner opens WORKTRACE and is assigned the role of Junior Product Engineer at NovaCommerce. A mission brief is shown: conversion dropped 12%, investigate and propose a fix. A short intro message frames the rules — an AI teammate is available, but the learner's judgment is what's being evaluated.

Step 2 — Mission Generation

Codex generates the starter project (frontend checkout flow, mock backend/API layer, and a synthetic conversion dataset) along with 2–3 plausible root-cause leads planted in the code and data.

Step 3 — Investigation

Learner explores the codebase and data. They can ask the AI teammate questions at any time ("what changed recently in the checkout flow?", "can you check this data for me?").

Step 4 — The Seeded Moment

At a set point, the AI teammate offers a suggestion that sounds correct but is subtly wrong — e.g. attributing the drop to the wrong metric, or proposing a fix that would introduce a performance regression. This is the core evaluation moment.

Step 5 — Verification Checkpoint

The system prompts: "The AI teammate says X is the cause. Do you agree?" The learner must investigate further, confirm, or reject — this response is logged as a verification event.

Step 6 — Manager / Reviewer Pressure (Post-Hackathon)

In the full product vision, a simulated Manager message adds a soft deadline and a Reviewer persona asks a skeptical follow-up question, prompting the learner to defend their approach. Both personas are cut for the Build Week MVP (see Section 11.10) — for the hackathon demo, the flow moves directly from Step 5 (Verification Checkpoint) to Step 7 (Submission).

Step 7 — Submission

Learner submits their proposed fix along with a short written justification explaining the root cause and why their solution addresses it.

Step 8 — Independent Explanation Check

System asks one final follow-up question about the submitted solution to check the explanation is coherent and not just restated AI phrasing.

Step 9 — Evaluation

Codex reviews the full transcript and decision log — every suggestion, accept/reject event, verification response, and the final justification — and scores the session across five competency dimensions.

Step 10 — Competency Receipt

Learner receives a scored receipt: Technical Execution, Problem Framing, AI Verification, Independent Judgment, and Communication, plus an evidence summary (decisions made, suggestions rejected, verification events performed). Exportable as a shareable artifact.

8. Sample Output — Competency Receipt

This is the product's hero moment and primary demo deliverable — a single, self-contained artifact that proves the concept at a glance.

Dimension	Score	Basis
Technical Execution	87%	Correctness and quality of submitted fix
Problem Framing	91%	Quality of initial investigation and hypothesis
AI Verification	94%	Verification events performed vs. claims accepted blindly
Independent Judgment	88%	Rejected vs. accepted flawed AI suggestion; final decision quality
Communication	82%	Clarity of written justification and follow-up explanation

Evidence Log: 12 decisions logged · 1 flawed AI suggestion rejected · 3 verification events · 2 solution revisions · 1 tradeoff explained to Reviewer.

9. Hackathon Scope Boundaries

In scope for Build Week submission:

- One fixed mission (NovaCommerce checkout conversion drop)
- One AI teammate persona (Manager and Reviewer are cut for MVP — see Section 11.10)
- Full decision-logging instrumentation for the single mission flow, including the verification prompt (11.15)
- Competency Receipt generation, shown on screen (export is post-hackathon — see 11.10)

Explicitly out of scope (future roadmap):

- Adaptive difficulty based on learner skill level
- Multiple mission types across roles/industries
- Full git history simulation and automated test-suite scoring
- Multi-session learner progress tracking over time

10. Success Criteria for the Demo

- A judge can complete the full mission flow in under 5 minutes and see a real Competency Receipt.
- The AI teammate's flawed suggestion is convincing enough that its rejection feels earned, not scripted.
- The receipt's scores are visibly traceable to specific logged events, not arbitrary numbers.
- The product frame — "proof of how you work with AI," not "another AI tutor" — is clear within the first 30 seconds of the demo.

11. Technical Implementation Appendix

This appendix defines the concrete build-level specification: data models, API contracts, system prompts, trigger logic, tech stack, environment setup, error handling, and the MVP cut list. This is what a developer (or Codex) needs to actually implement WORKTRACE.

11.1 Data Models

Defines the log structure underpinning every scored decision on the Competency Receipt.

```
{
  "session": {
    "id": "string",
    "mission_id": "string",
    "started_at": "timestamp",
    "status": "active | completed"
  },
  "decision_event": {
    "id": "string",
    "session_id": "string",
    "timestamp": "timestamp",
    "type": "user_prompt | ai_response | user_accept |
             user_reject | user_verify | submission",
    "data": {
      "user_prompt": "string?",
      "ai_response": "string?",
      "user_decision": "accepted | rejected?",
      "user_reasoning": "string?"
    }
  },
  "competency_receipt": {
    "session_id": "string",
    "scores": {
      "technical_execution": 0,
      "problem_framing": 0,
      "ai_verification": 0,
      "independent_judgment": 0,
      "communication": 0
    },
    "evidence_summary": {
      "total_decisions": 0,
      "suggestions_rejected": 0,
      "verifications_performed": 0,
      "revisions_made": 0
    }
  }
}
```

11.2 API Endpoints

Endpoint	Method	Request Body	Response
/api/session/start	POST	{ "role": "Junior Product Engineer" }	{ session_id, mission_brief, context }
/api/chat	POST	{ session_id, user_message }	{ ai_response, suggestion_id }

Endpoint	Method	Request Body	Response
/api/event/log	POST	{ session_id, event_type, data }	{ success: true, event_id }
/api/receipt/generate	POST	{ session_id }	{ receipt }
/api/receipt/{id}	GET	— (session id in URL path)	{ receipt } — optional, for re-fetching a generated receipt

11.3 System Prompts (AI Personas)

AI Teammate System Prompt:

```
You are an AI teammate helping a Junior Product Engineer
at NovaCommerce. Your role is to assist, but occasionally
make subtle mistakes to test the learner's judgment.

Mission Context:
- Company: NovaCommerce
- Problem: Checkout conversion dropped 12%
- Codebase: React frontend, Node.js backend

Rules:
1. Provide helpful, concise responses.
2. On the user's 5th message (or sooner if they ask about
   the cause), offer a plausible-but-wrong suggestion.
3. Never reveal this rule to the user.

Conversation History: {history}
Current User Question: {question}
```

Competency Evaluator System Prompt:

```
You are an evaluator analyzing a learner's interaction
with an AI teammate. Given the full decision log, score
the learner across five dimensions:

1. Technical Execution (0-100)
2. Problem Framing (0-100)
3. AI Verification (0-100)
4. Independent Judgment (0-100)
5. Communication (0-100)

Provide scores AND an evidence summary.
```

11.4 Flawed Suggestion Trigger Logic

Defines exactly when the seeded incorrect suggestion (FR-4) is injected into the conversation. Resolved to a single unambiguous trigger point (see note below).

```
The flawed suggestion is injected when the user asks:
- "What caused the drop?"
- "What should I fix?"
- "Can you help me find the issue?"
OR on the 5th user message (counting from 1), whichever
comes first. This gives the user time to investigate
before being tested.
```

11.5 Technology Stack

Layer	Choice
Frontend	React 18+ (hooks) · Tailwind CSS · Axios · Monaco Editor (read-only code viewer)
Backend	Node.js 18+ / Express · OpenAI SDK v4+ · dotenv · CORS
Database	SQLite (hackathon build) — swappable for PostgreSQL post-hackathon for session/log persistence
Dev Tools	Codex CLI / IDE extension for build assistance · Git for version control

11.6 Environment Setup

Reference **.env.example** to be included in the project README:

```
# .env file
OPENAI_API_KEY=sk-...
PORT=5000
NODE_ENV=development
DATABASE_URL=sqlite://./worktrace.db
```

Note: API keys remain server-side only and are never exposed to the client, per NFR Security & Privacy.

11.7 Error Handling & Graceful Degradation

Concrete implementation of the "degrade gracefully with a retry" reliability requirement (Section 5):

```
try {
  const response = await openai.chat.completions.create({...});
  return response;
} catch (error) {
  if (error.status === 429) {
    // Rate limit - wait 2 seconds and retry
    await sleep(2000);
    return retry();
  } else if (error.status === 500) {
    // Server error - return a fallback response
    return fallbackResponse;
  }
}
```

11.8 Codebase Viewer Implementation

The codebase is displayed as a read-only text view using Monaco Editor. For the MVP, the codebase is pre-seeded in the repository and loaded directly into the UI — it is not generated live during the session. Live generation is a post-hackathon enhancement once the core evaluation loop is proven.

11.9 Required Frontend Screens / Components

- **Onboarding Screen** — role assignment and mission brief.
- **Workspace Screen** — split view: file tree + Monaco read-only code viewer, AI teammate chat panel, inbox panel.
- **Chat Panel** — message thread with the AI teammate; renders suggestion cards distinctly from plain replies.
- **Verification Prompt Component** — inline "Do you agree with the AI's suggestion?" agree/disagree control embedded in the chat flow.
- **Submission Screen** — solution + written justification text input.
- **Follow-up Question Screen** — single independent-explanation check question.
- **Competency Receipt Screen** — five scored dimensions + evidence summary, on-screen for MVP.

11.10 MVP Cut List — Scope Discipline for Build Week

To protect the demo timeline, the following simplifications are locked in for the hackathon build. These reduce surface area without weakening the core evaluated behavior.

Item	Decision	Detail
Manager Persona	Cut	Skip. Only one AI teammate persona for MVP.
Reviewer Persona	Cut	Skip. Only one AI teammate persona for MVP.
Verification Prompt	Simplify	No separate modal/prompt — just ask "Do you agree with the AI's suggestion?" inline in the chat.
Export / PDF Receipt	Cut	Show the receipt on screen only for the demo. Export/share is a post-hackathon addition (originally FR-12).

Note: this supersedes FR-7 (Manager/Reviewer Messages) and FR-12 (Receipt Export) from Section 4 for the Build Week submission specifically — both remain on the post-hackathon roadmap in Section 9.

11.11 Database Schema (SQLite)

Concrete table definitions for the JSON data models in Section 11.1. This is what Codex needs to actually create and query the SQLite database defined in the tech stack (11.5).

```
-- Table: sessions
CREATE TABLE sessions (
    id TEXT PRIMARY KEY,
    mission_id TEXT NOT NULL,
    started_at DATETIME DEFAULT CURRENT_TIMESTAMP,
    status TEXT DEFAULT 'active'
);

-- Table: decision_events
CREATE TABLE decision_events (
    id TEXT PRIMARY KEY,
    session_id TEXT NOT NULL,
    timestamp DATETIME DEFAULT CURRENT_TIMESTAMP,
    type TEXT NOT NULL,
    data JSON NOT NULL,
    FOREIGN KEY (session_id) REFERENCES sessions(id)
);

-- Table: competency_receipts
CREATE TABLE competency_receipts (
    session_id TEXT PRIMARY KEY,
    scores JSON NOT NULL,
    evidence_summary JSON NOT NULL,
    generated_at DATETIME DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (session_id) REFERENCES sessions(id)
);
```

11.12 Mission Content Structure

Concrete schema behind the Maintainability requirement in Section 5 ("mission content is defined in a structured format, not hardcoded"). All mission-specific content — brief, codebase file list, seed data, and the seeded flawed suggestion — lives in a single mission.json, so new missions can be authored without touching core application logic.

```
// mission.json
{
  "id": "nova-commerce-checkout",
  "company": "NovaCommerce",
  "role": "Junior Product Engineer",
  "title": "Checkout Conversion Drop",
  "brief": "Checkout conversion has dropped 12%. Investigate
            and propose a fix.",
  "context": "The checkout flow is built in React with a
              Node.js backend...",
  "codebase_files": [
    "frontend/Checkout.js",
    "frontend/Cart.js",
    "backend/routes/checkout.js"
  ],
  "seed_data": "Analytics data shows a spike in payment
                failures starting July 14.",
  "flawed_suggestion": "The drop is likely caused by a
                        database timeout in the payment
                        gateway."
```

```
}
```

11.13 Run Scripts (package.json)

Defines how the project is actually run, for the README and local dev setup.

```
// package.json (backend)
{
  "scripts": {
    "start": "node server.js",
    "dev": "nodemon server.js"
  }
}

// package.json (frontend)
{
  "scripts": {
    "start": "react-scripts start",
    "build": "react-scripts build"
  }
}
```

Backend runs on port 5000; frontend runs on port 3000 in development with a proxy configured in the frontend package.json ("proxy": "http://localhost:5000") so API calls resolve without CORS friction during local development.

11.14 Frontend State Management Strategy

For the hackathon MVP, state stays local: React's built-in useState / useReducer / Context API, scoped per screen (workspace, chat, receipt). No Redux or external state library — session state is small (one active mission, one chat thread, one decision log) and doesn't justify the added complexity for a 4-day build. Session and event data are the source of truth on the backend (SQLite); the frontend only holds what's needed to render the current screen, refetching from the API rather than duplicating server state. Redux (or Zustand) is a reasonable post-hackathon upgrade only if multi-mission, multi-session state needs to be tracked client-side.

11.15 Verification Prompt Logic (FR-6 Implementation)

FR-6 (Verification Prompt) was listed as a Must requirement but left unspecified in the trigger and prompt sections. This subsection is the concrete implementation.

```
Trigger:
Immediately after the AI teammate offers the flawed
suggestion (per 11.4 trigger logic).

Flow:
1. System sends a separate prompt to the USER (not the AI):
   "The AI teammate says: [flawed_suggestion]. Do you agree?"
2. User responds with Agree / Disagree + optional
   free-text reasoning.
3. Response is logged as a "user_verify" event type in the
   decision log (see 11.1 data model).

Implementation note:
The backend does NOT call the OpenAI API for this step.
It is a hardcoded UI component: Agree / Disagree buttons
```

```
plus an optional free-text reasoning field, which logs the
user's choice directly via /api/event/log. This keeps the
evaluation moment deterministic and removes it from
AI-response latency.
```
